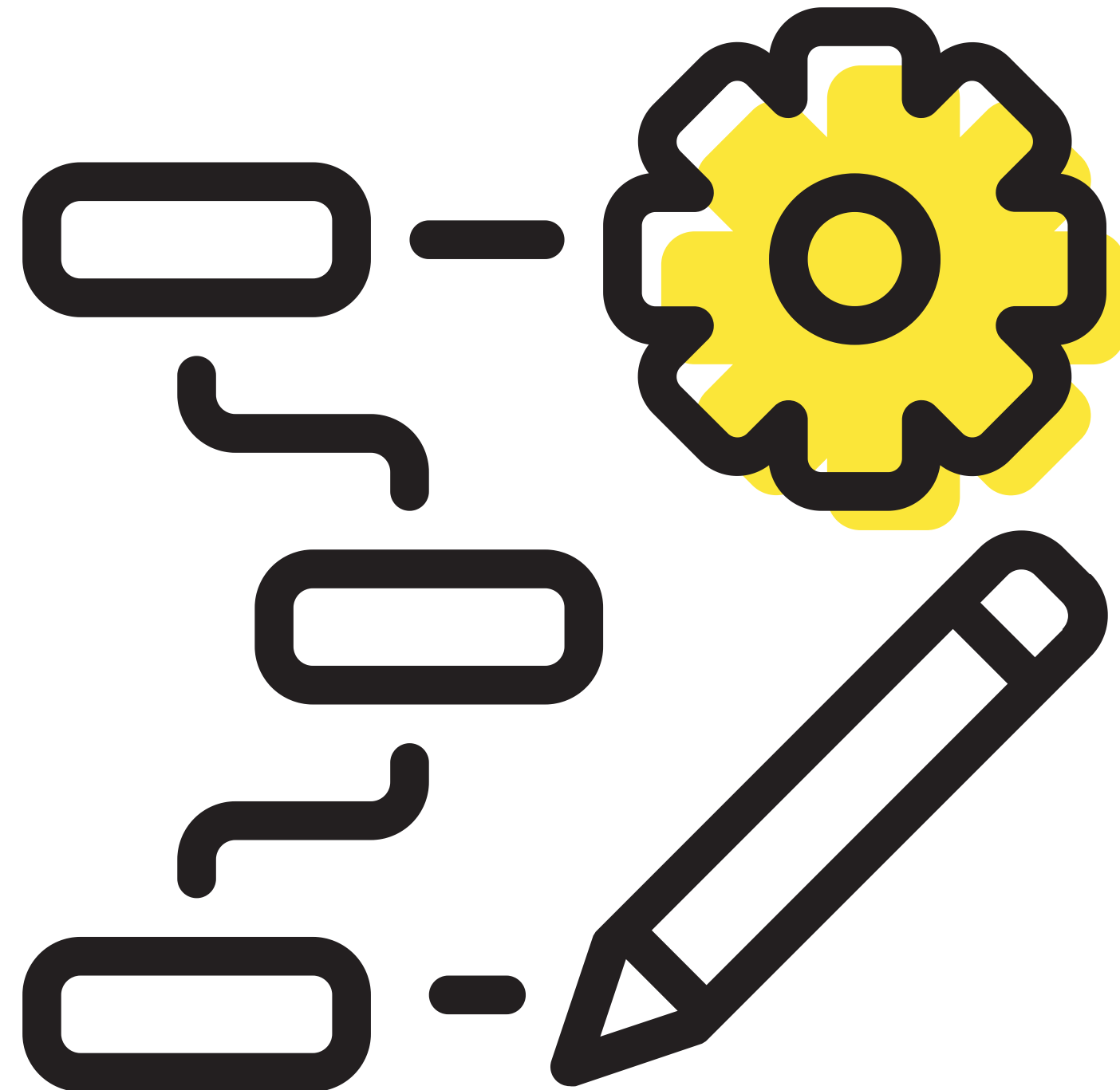
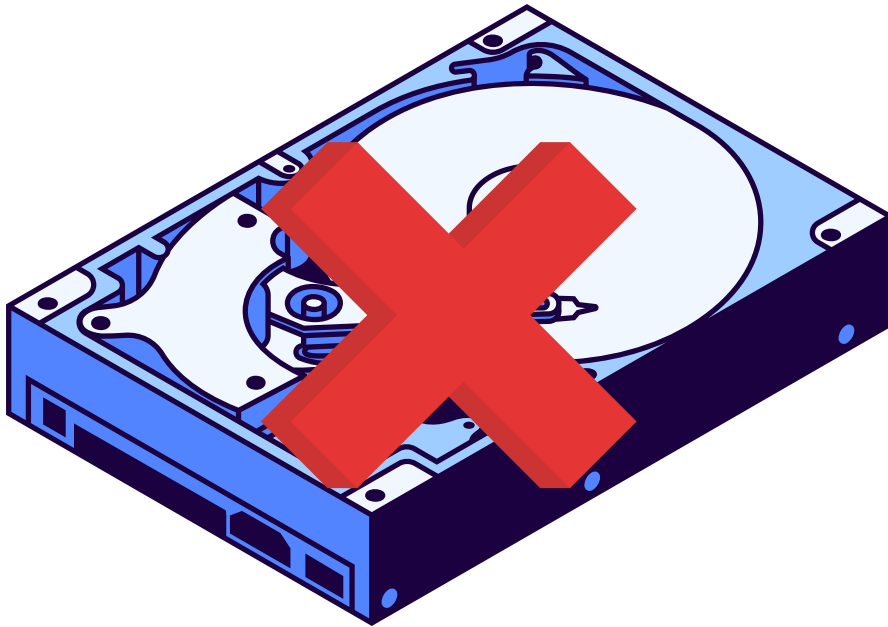


Saé 1.05 - Outil de reporting pour la detection de gros fichiers

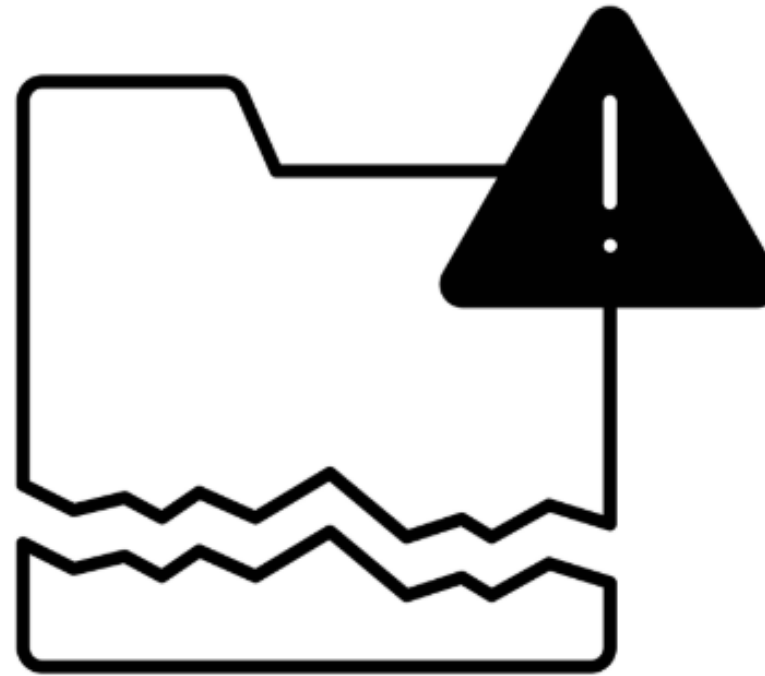


Objectif du projet

Manque de stockage



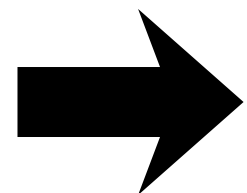
Perte de donnée



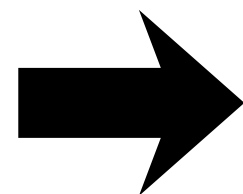
Besoin de l'outil



Meilleure visibilité des données



Meilleure visibilité du manque de stockage

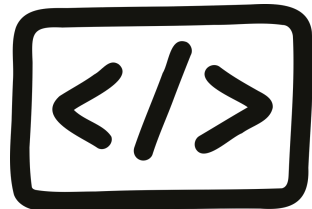
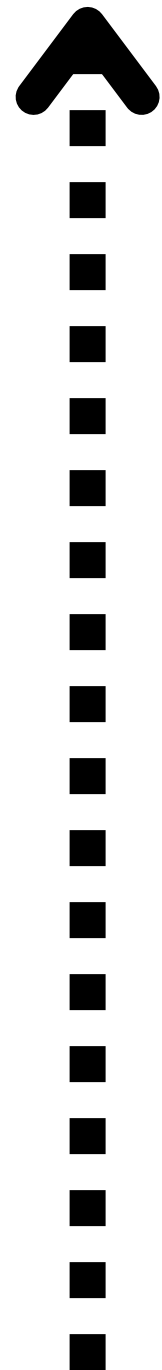


Pas de perte de données en cas de mise à jour ou autres

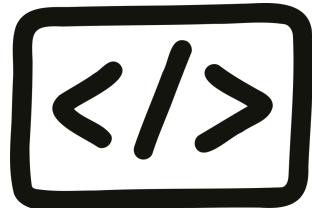
Architecture générale de la solution



PowerShell



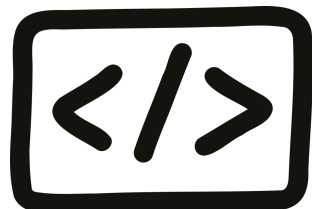
Script Python 1 : Sélection du répertoire



Script Python 2 : Analyse & génération JSON



Fichier JSON (données intermédiaires)



Script Python 3 : Interface graphique & suppression

Script 1

Bibliothèque

```
SAE-105-script1.py 1 X
C: > Users > ZOSD > saé > SAE-105-script1.py >
1  from PyQt5.QtWidgets import QApplication, QFileDialog
2  from pathlib import Path
3  import sys
4
5  def choisir_repertoire():
6      app = QApplication(sys.argv)
7      rep = QFileDialog.getExistingDirectory(None, "Sélectionnez un répertoire à analyser", str(Path.home()))
8      print(rep if rep else "")
9      sys.exit(0)
10
11 if __name__ == "__main__":
12     choisir_repertoire()
```

- Garantit que la fonction ne s'exécute que si le fichier est lancé directement.
- Empêche l'exécution automatique si ce script est importé dans un autre module.

- Initialise l'environnement graphique PyQt.

- lancer une mini-application graphique,
- ouvrir une fenêtre pour choisir un répertoire,
- afficher le chemin choisi dans la console.

Script 2

```
1  ✓ from pathlib import Path
2  import json
3  import sys
4
5  ✓ def analyse_arborescence(repertoire_base):
6      rep = Path(repertoire_base)
7      liste_fichiers = []
8  ✓  for fichier in rep.rglob("*"):
9  ✓      if fichier.is_file():
10         chemin = str(fichier.resolve())
11         taille = fichier.stat().st_size
12         liste_fichiers.append([chemin, taille])
13     return liste_fichiers
14
15  ✓ def trier_par_taille(liste_fichiers):
16     return sorted(liste_fichiers, key=lambda x: x[1], reverse=True)
17
18  ✓ def filtrer_fichiers(liste_fichiers, taille_mini_mib, nb_max):
19     taille_mini_octets = taille_mini_mib * 1048576
20     liste_filtree = [f for f in liste_fichiers if f[1] >= taille_mini_octets]
21     return liste_filtree[:nb_max]
22
23  ✓ def sauvegarder_json(liste_fichiers, nom_fichier_json):
24     liste_corrigee = []
25  ✓  for chemin, taille in liste_fichiers:
26         chemin_corrige = chemin.replace("\\", "\\")
27         liste_corrigee.append([chemin_corrige, taille])
28  ✓  with open(nom_fichier_json, "w", encoding="utf-8") as f:
29         json.dump(liste_corrigee, f, indent=4)
```

→ parcourir tous les fichiers,
→ récupérer chemin + taille,
→ créer la liste des fichiers.

→ trier du plus gros au plus petit

→ convertir MiB → octets,
→ garder seulement les fichiers assez gros,
→ limiter le nombre.

→ corriger les chemins (antislash
→ double antislash),
→ créer une liste propre,
→ écrire dans un fichier JSON.

Script 2

```
30
31 if __name__ == "__main__":
32     if len(sys.argv) >= 2:
33         repertoire_de_base = sys.argv[1]
34     else:
35         repertoire_de_base = r"C:\INSTALL"
36
37     TAILLE_MINI_MIB = 10
38     NB_MAXI_FICHIERS = 100
39
40     liste = analyse_arborescence(repertoire_de_base)
41     liste = trier_par_taille(liste)
42     liste = filtrer_fichiers(liste, TAILLE_MINI_MIB, NB_MAXI_FICHIERS)
43     sauvegarder_json(liste, "gros_fichiers.json")
```

→ exécuter le script principal.

→ récupérer le dossier passé en argument.

→ seuil minimum : 10 MiB.

→ limiter à 100 fichiers max.

→ enchaîner les étapes pour générer le fichier final.

Script 3

C: > Users > ZQSD > saé > SAE-105-script3.py > creation_script_suppression

```
1  import json
2  import random
3  from pathlib import Path
4
5  from PyQt5.QtWidgets import QApplication
6  from PyQt5.QtGui import QColor
7
8  from Creation_Onglets import Onglets
9  from Creation_Camembert import Camembert
10 from Creation_Legendes import Legendes
11 from Creation_Boutons import Boutons
12
13 NB_LEGENDES_PAR_PAGE = 25
14 NB_MAXI_FICHIERS = 100
15
16 def lire_json(nom_fichier_json):
17     with open(nom_fichier_json, "r", encoding="utf-8") as f:
18         return json.load(f)
19
20 def generer_couleurs(nb_couleurs):
21     liste_couleurs = []
22     for _ in range(nb_couleurs):
23         r = random.randint(0, 255)
24         g = random.randint(0, 255)
25         b = random.randint(0, 255)
26         liste_couleurs.append(QColor(r, g, b))
27     return liste_couleurs
28
29 def creation_script_suppression():
30     global liste_legendes, liste_fichiers
```

→ charger les modules graphiques et les composants personnalisés.

→ afficher 25 fichiers par onglet.

→ limiter l'affichage à 100 fichiers max.

→ lire les données analysées depuis le fichier JSON.

→ créer des couleurs aléatoires pour le camembert.

Script 3

C: > Users > ZQSD > saé > SAE-105-script3.py > creation_script_suppression

```
29 def creation_script_suppression():
30
40     index_fichier = 0
41     for page in liste_legendes:
42         etats = page.recupere_etats_case_a_cocher()
43         for coche in etats:
44             if coche:
45                 chemin = liste_fichiers[index_fichier][0]
46                 chemin = chemin.replace("\\", "/")
47                 lignes_script.append(f"Remove-Item -Path \"{chemin}\" -Force")
48                 index_fichier += 1
49
50     lignes_script.append("    } else { Write-Output \"Opération annulée...\" }")
51     lignes_script.append("} else { Write-Output \"Opération annulée...\" }")
52
53     with open("suppression_fichiers.ps1", "w", encoding="utf-8") as f:
54         for ligne in lignes_script:
55             f.write(ligne + "\n")
56
57     print("Script PowerShell généré : suppression_fichiers.ps1")
58
59 if __name__ == "__main__":
60     repertoire_base = r"C:\INSTALL"
61     liste_fichiers = lire_json("gros_fichiers.json")
62     liste_couleurs = generer_couleurs(NB_MAXI_FICHIERS)
```

→ récupérer les fichiers cochés,
→ générer les commandes Remove-Item,
→ écrire le script PowerShell.

→ rendre le chemin compatible Windows/Linux.

→ créer le fichier de suppression.

Script 3

```
C: > Users > ZQSD > saé > SAE-105-script3.py > creation_script_suppression
```

```
64 app = QApplication([])
65 fenetre = Onglets()
66
67 fromage = Camembert(liste_fichiers, liste_couleurs)
68 layout_fromage = fromage.dessine_camembert()
69 fenetre.add_onglet("Camembert", layout_fromage)
70
71 liste_legendes = []
72 nb_pages = (len(liste_fichiers) + NB_LEGENDES_PAR_PAGE - 1) // NB_LEGENDES_PAR_PAGE
73
74 for num_page in range(nb_pages):
75     start = num_page * NB_LEGENDES_PAR_PAGE
76     leg = Legendes(liste_fichiers, liste_couleurs, start)
77     liste_legendes.append(leg)
78     layout_leg = leg.dessine_legendes()
79     fenetre.add_onglet(f"Légende {num_page + 1}", layout_leg)
80
81 ihm = Boutons(repertoire_base, creation_script_suppression)
82 layout_ihm = ihm.dessine_boutons()
83 fenetre.add_onglet("IHM", layout_ihm)
84
85 fenetre.show()
86 app.exec_()
```

→ lancer l'application graphique.
Camembert()

→ créer le graphique circulaire.

→ créer les cases à cocher pour chaque fichier.

→ ajouter le bouton pour générer le script PowerShell.

→ ajouter chaque onglet dans l'IHM.

→ démarrer l'interface graphique.

Fichier JSON

```
C: > Users > ZQSD > saé > {} gros_fichiers.json > ...
```

```
1  [
2      [
3          "C:\\\\Users\\\\furka\\\\BUT-RT\\\\EVA 2 (VM)\\\\EVE_SAE12.ova",
4          4443034624
5      ],
6      [
7          "C:\\\\Users\\\\furka\\\\BUT-RT\\\\machine virtuelle\\\\Debian 12.ova",
8          3268134400
9      ]
10 ]
```

- contient les chemins + tailles des fichiers volumineux,
- sert à afficher le camembert et les légendes,
- est recréé à chaque nouvelle analyse.

Fichier création du bouton

C: > Users > ZQSD > saé > Creation_Boutons.py > ...

```
1 from PyQt5.QtWidgets import QVBoxLayout, QPushButton, QLabel
```

```
3 class Boutons:
```

```
4     def __init__(self, repertoire_base, callback):
```

```
5         self.repertoire_base = repertoire_base
```

```
6         self.callback = callback
```

```
8     def dessine_boutons(self):
```

```
9         layout = QVBoxLayout()
```

```
11         label = QLabel(f"Répertoire analysé : {self.repertoire_base}")
```

```
12         layout.addWidget(label)
```

```
14         bouton = QPushButton("Générer le script PowerShell de suppression")
```

```
15         bouton.clicked.connect(self.callback)
```

```
16         layout.addWidget(bouton)
```

```
18     return layout
```

- QVBoxLayout: un layout vertical.
- QPushButton : un bouton cliquable.
- QLabel : un simple texte affiché dans l'interface

- But de la classe : construire un petit panneau
- repertoire_base : le chemin qui sera affiché dans le label
- callback : une fonction à appeler quand on clique sur le bouton
- Ces deux valeurs sont stockées dans l'objet (self.repertoire_base, self.callback) pour être réutilisées dans dessine_boutons.

- Crée un layout vertical qui va contenir le label puis le bouton, l'un en dessous de l'autre.

- Crée un bouton
- Connecte le signal clicked du bouton à la fonction callback fournie au constructeur.

C: > Users > ZQSD > saé > Creation_Camembert.py > Camembert > dessine_camembert

```
1  from PyQt5.QtWidgets import QVBoxLayout
2  from PyQt5.QtChart import QChart, QChartView, QPieSeries
3  import os
4
5  class Camembert:
6      def __init__(self, liste_fichiers, liste_couleurs):
7          self.liste_fichiers = liste_fichiers
8          self.liste_couleurs = liste_couleurs
9
10     def dessine_camembert(self):
11         layout = QVBoxLayout()
12         series = QPieSeries()
13
14         for i, (chemin, taille) in enumerate(self.liste_fichiers):
15             if taille > 0:
16                 nom_fichier = os.path.basename(chemin)
17                 taille_mb = round(taille / (1024 * 1024), 2)
18                 label = f"{nom_fichier} - {taille_mb} MB"
19
20                 slice = series.append(label, taille)
21                 slice.setBrush(self.liste_couleurs[i])
22                 slice.setLabelVisible(True)
23
24         chart = QChart()
25         chart.addSeries(series)
26         chart.setTitle("Répartition des fichiers (en MB)")
27         chart.legend().hide()
28
29         chart_view = QChartView(chart)
30         layout.addWidget(chart_view)
31
32         return layout
```

Création du camembert

→ créer un graphique circulaire,
→ afficher les tailles des fichiers en MB,
→ utiliser des couleurs aléatoires.

→ construire le camembert avec PyQt5,
→ afficher les noms + tailles,
→ retourner le layout pour l'IHM.

Création des légendes

C: > Users > ZQSD > saé > Creation_Legendes.py > Legendes > dessine_legendes

```
1  from PyQt5.QtWidgets import QVBoxLayout, QHBoxLayout, QLabel, QCheckBox
2  from PyQt5.QtGui import QColor
3
4  NB_LEGENDES_PAR_PAGE = 25
5
6
7  class Legendes:
8      def __init__(self, liste_fichiers, liste_couleurs, index_depart):
9
10         self.liste_fichiers = liste_fichiers
11         self.liste_couleurs = liste_couleurs
12         self.index_depart = index_depart
13         self.cases = []
14
15     def dessine_legendes(self):
16
17         layout = QVBoxLayout()
18
19         for i in range(self.index_depart, min(self.index_depart + NB_LEGENDES_PAR_PAGE, len(self.liste_fichiers))):
20             chemin, taille = self.liste_fichiers[i]
21             couleur = self.liste_couleurs[i]
22
23             ligne = QHBoxLayout()
24
25             label_couleur = QLabel(" ")
26             label_couleur.setStyleSheet(f"background-color: {couleur.name()}; border: 1px solid black;")
27             ligne.addWidget(label_couleur)
28
29             label_nom = QLabel(chemin)
30             ligne.addWidget(label_nom)
31
32             case = QCheckBox()
33             self.cases.append(case)
34             ligne.addWidget(case)
35
36             layout.addLayout(ligne)
```

→ créer les cases à cocher pour chaque fichier,
→ afficher le nom + couleur associée,
→ gérer 25 fichiers par page.

→ construire le layout avec PyQt5,
→ afficher les légendes dans l'IHM.

Création des legendes

```
37  
38     return layout  
39  
40     def recupere_etats_case_a_cocher(self):  
41  
42         return [case.isChecked() for case in self.cases]
```

- récupérer l'état de chaque case (cochée ou non),
 - renvoyer une liste de booléens.

Analyse gros fichier

C: > Users > ZOSD > saé > > Analyse Gros Fichiers.ps1

```
1 $python = ".\venv\Scripts\python.exe"
```

→ utiliser le Python du projet (en local).

```
2  
3 $rep_base = & $python .\SAE-105-script1.py
```

→ lancer la sélection du dossier via l'IHM.

```
4  
5 if (-not $rep_base) {  
6     Write-Host "Aucun répertoire sélectionné. Arrêt du script."  
7     exit  
8 }  
9
```

→ vérifier que l'utilisateur a bien choisi un dossier.

```
10 if (-not (Test-Path $rep_base)) {  
11     Write-Host "Le répertoire sélectionné n'existe pas."  
12     exit  
13 }  
14
```

→ vérifier que le dossier existe vraiment

```
15 Write-Host "Répertoire sélectionné : $rep_base"
```

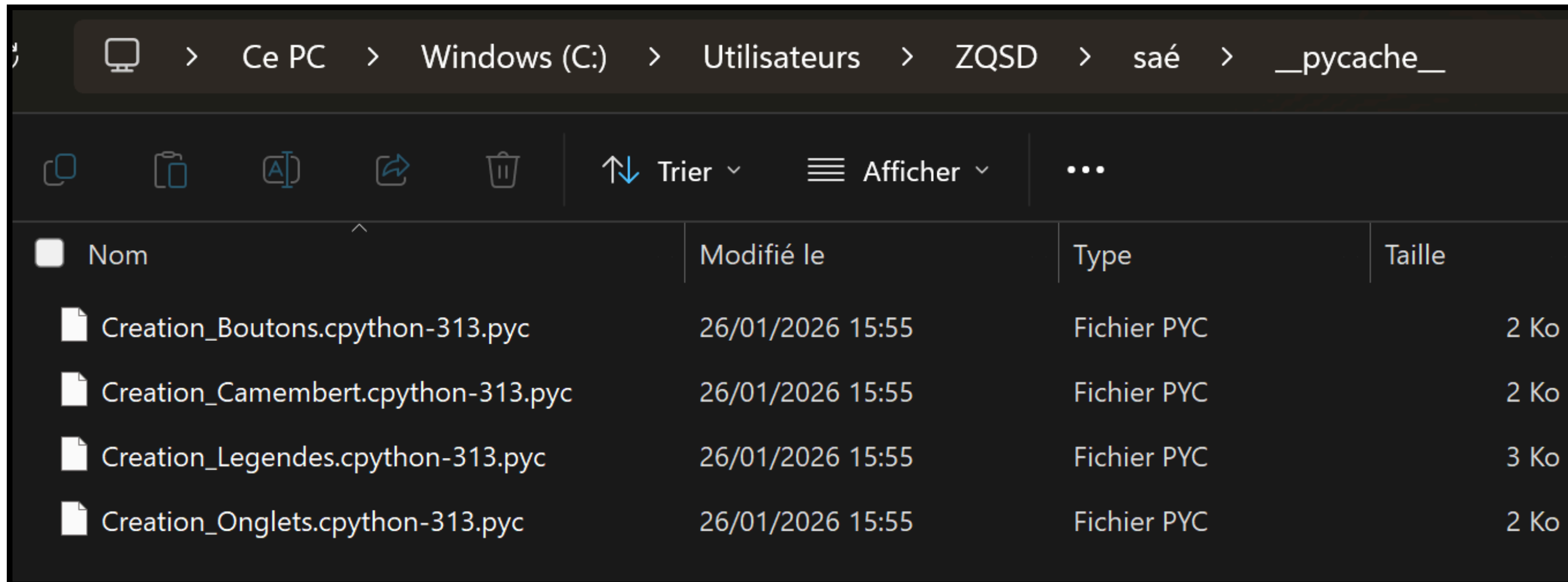
```
16 & $python .\SAE-105-script2.py $rep_base
```

→ lancer l'analyse des fichiers.

```
17  
18  
19 & $python .\SAE-105-script3.py
```

→ lancer l'IHM avec camembert + légendes.

Fichier annexe



Ce PC > Windows (C:) > Utilisateurs > ZQSD > saé > __pycache__			
Trier ▾ Afficher ▾ ⋮			
Nom	Modifié le	Type	Taille
Creation_Boutons.cpython-313.pyc	26/01/2026 15:55	Fichier PYC	2 Ko
Creation_Camembert.cpython-313.pyc	26/01/2026 15:55	Fichier PYC	2 Ko
Creation_Legendes.cpython-313.pyc	26/01/2026 15:55	Fichier PYC	3 Ko
Creation_Onglets.cpython-313.pyc	26/01/2026 15:55	Fichier PYC	2 Ko

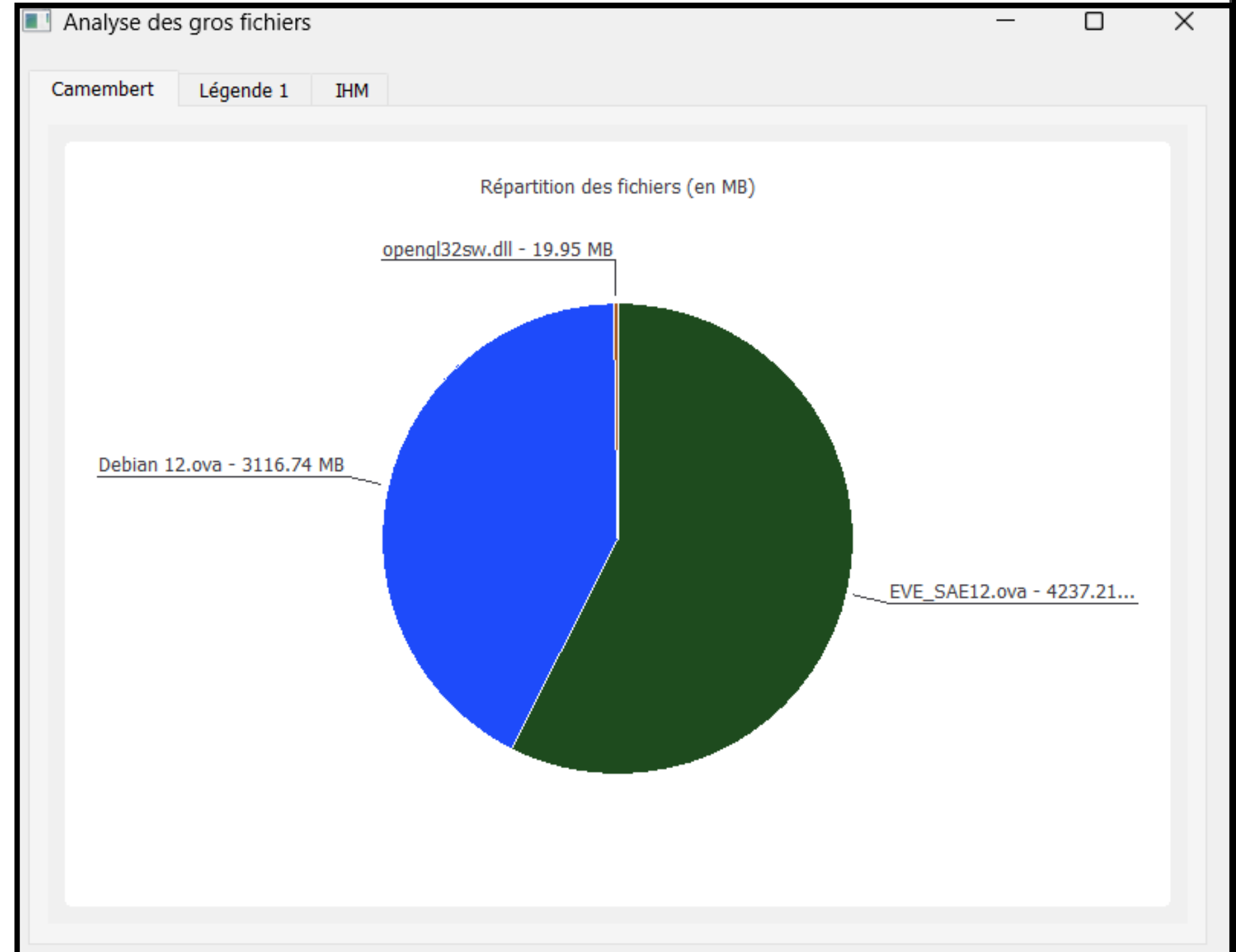
Fichier	Signification
Creation_Boutons.cpython-313.pyc	Version compilée du fichier Creation_Boutons.py
Creation_Camembert.cpython-313.pyc	Version compilée de Creation_Camembert.py
Creation_Legendes.cpython-313.pyc	Version compilée de Creation_Legendes.py
Creation_Onglets.cpython-313.pyc	Version compilée de Creation_Onglets.py

Tests et résultats obtenus



Répertoire analysé : C:\\INSTALL

Générer le script PowerShell de suppression



Conclusion